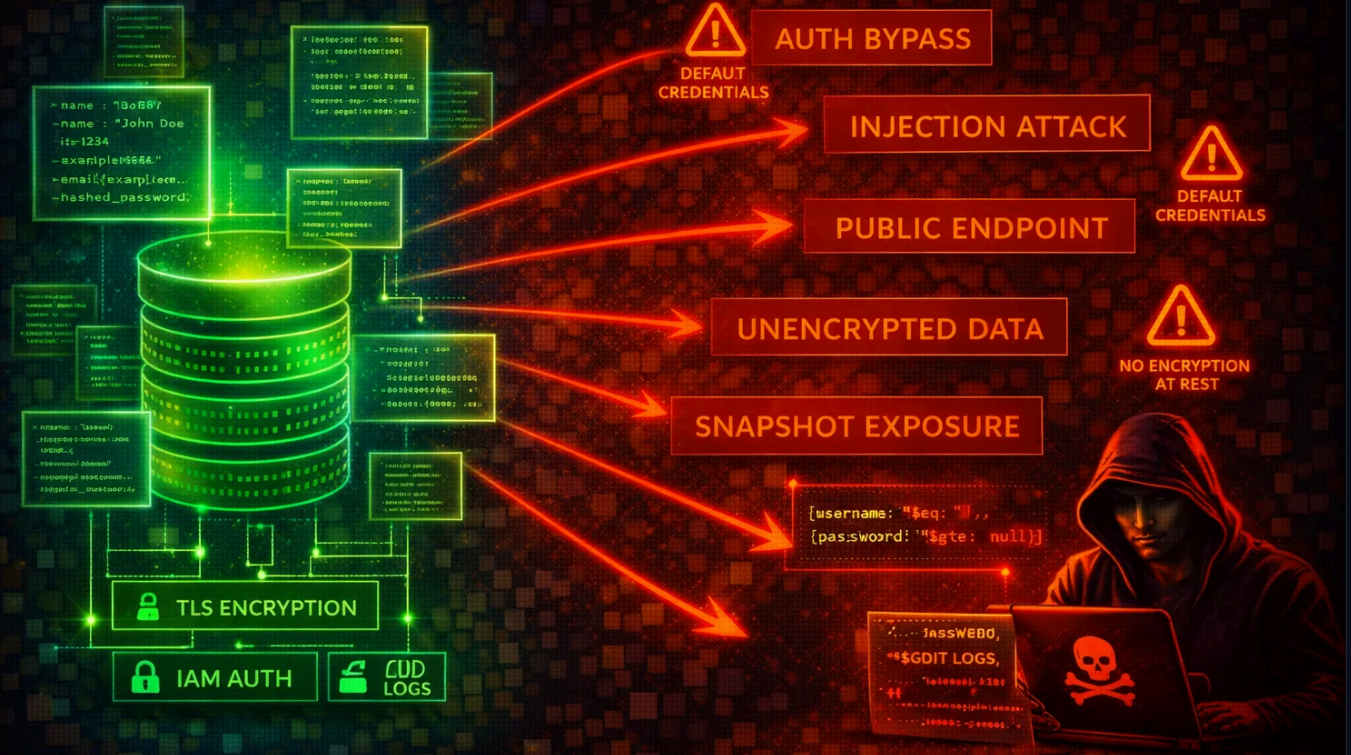




Amazon DocumentDB Security

DATABASE

Service Overview

Amazon DocumentDB is a managed document database service compatible with the MongoDB wire protocol. It supports MongoDB 4.0, 5.0, and 8.0 compatibility modes. Clusters run exclusively inside a VPC with no public endpoint option.

From an attacker's perspective, DocumentDB is valuable because:

- It stores structured application data (user records, session data, financial documents) that is directly exploitable.
- The MongoDB wire protocol compatibility means NoSQL injection techniques that work against MongoDB generally work against DocumentDB (except those requiring server-side JavaScript execution, such as `$where`, which DocumentDB does not support).
- Snapshots can be shared publicly or cross-account, creating data exfiltration paths independent of network access.
- TLS and audit logging are configurable parameters that can be disabled by anyone with `rds:ModifyDBClusterParameterGroup` permissions (TLS change requires instance reboot and is visible in CloudTrail).
- Encryption at rest cannot be enabled after cluster creation — if a cluster was created unencrypted, that decision is permanent.

Security Risk Assessment

Factor	Rating	Detail
Data Sensitivity	High	Stores application-level structured data: PII, credentials, financial records
Blast Radius	High	Cluster-wide — all databases and collections in a cluster share the same auth, TLS, and encryption settings
Lateral Movement	Medium	Master credentials may be reused; snapshots enable data movement to attacker-controlled accounts
Detection Difficulty	Medium	Audit logging disabled by default; data-plane operations (queries) are not in CloudTrail
Recovery Complexity	Medium	Point-in-time restore available (up to 35 days), but encryption settings are immutable

✕ Attack Vectors

Initial Access

#	Vector	Description
1	NoSQL Injection via Application	Applications using MongoDB drivers against DocumentDB are vulnerable to operator injection (\$gt, \$ne, \$regex) if user input is not sanitized. Authentication bypass: {"username": {"\$ne": ""}, "password": {"\$ne": ""}} returns all documents. Note: \$where is not supported by DocumentDB (no server-side JavaScript execution).
2	Credential Theft from Secrets Manager / Environment	DocumentDB master credentials stored in environment variables, application configs, or Secrets Manager with overly broad access policies.
3	Snapshot Restore in Attacker Account	If a manual snapshot is shared with "all" (public) or a specific attacker account ID, the attacker can restore the full database in their own account and read all data without network access to the original cluster.
4	VPC Peering / Transit Gateway Pivot	DocumentDB has no public endpoint, but an attacker who compromises a peered VPC or Transit Gateway attachment gains network-level access to the cluster endpoint.
5	Parameter Group Tampering	An attacker with rds:ModifyDBClusterParameterGroup can set tls=disabled and audit_logs=disabled, downgrading security and reducing visibility after an instance reboot.

Persistence & Privilege Escalation

#	Vector	Description
1	Database User Creation	An attacker with master credentials can create new database users via the MongoDB shell (db.createUser()). These users persist independently of IAM and survive credential rotation of the master user.
2	Snapshot Exfiltration for Offline Access	create-db-cluster-snapshot + modify-db-cluster-snapshot-attribute to share snapshot with attacker account. Data access persists even after the original cluster is secured.
3	Cross-Region Snapshot Copy	copy-db-cluster-snapshot with --kms-key-id in a different region creates a durable copy of data outside the victim's operational region.

⚠ Misconfigurations

Critical

#	Misconfiguration	Impact
1	TLS Disabled (tls=disabled)	All data between application and cluster is cleartext. Credentials visible to anyone with access. Security Hub control: DocumentDB-TLS-Disabled
2	Audit Logging Disabled (default)	No record of DDL or DML operations. Audit failures, data access, and schema changes are invisible. Security Hub control: DocumentDB-Audit-Disabled
3	Public Snapshots (restore attribute set to all)	Any AWS account worldwide can restore and read all data. Security Hub control: DocumentDB-3. AWS Config rule: docdb-snapshot-public-prohibited
4	Encryption at Rest Not Enabled	Data on disk, automated backups, snapshots, and replicas are all unencrypted. Cannot be enabled — cluster must be recreated. Security Hub control: DocumentDB.1
5	Deletion Protection Disabled	Cluster can be deleted by any principal with rds:DeleteDBCluster. Security Hub control: DocumentDB.5

High

#	Misconfiguration	Impact
1	Default KMS Key Used for Encryption	The AWS managed key (aws/rds) is a shared cross-account and cannot have custom policies. Limits the ability to control encryption of encrypted snapshots.
2	Backup Retention Below 7 Days	Reduces the window for point-in-time recovery after a breach. Security Hub control: DocumentDB.2 (minimum 7 days recommended)
3	Overly Permissive Security Groups	Security group allowing 0.0.0.0/0 within the VPC. While DocumentDB has no public endpoint, this allows any resource in the VPC (including compromised instances) to connect to the cluster.
4	No Profiler Enabled (profiler=disabled)	Slow or suspicious queries (large collections, regex-based exfiltration) go undetected. Threshold is 100ms when enabled.
5	Master Credentials Not Rotated	Static master username/password with no Secrets Manager rotation configured. Credentials may be embedded in application code or CLI scripts.

4 **Event
Subscription for
Reconnaissance**

`create-event-subscription` with an attacker-controlled SNS topic receives notifications about cluster modifications, failovers, and maintenance — continuous intel on the target environment.

5 **Disable Deletion
Protection**

`modify-db-cluster --no-deletion-protection` removes the safety net, enabling future destructive actions including cluster deletion (ransomware path).

🔍 Enumeration Commands

All commands use the `aws docdb` CLI namespace. Verified against AWS CLI v2.

List All DocumentDB Clusters

```
aws docdb describe-db-clusters \
  --filter Name=engine,Values=docdb
```

List All DocumentDB Instances

```
aws docdb describe-db-instances
```

Check TLS and Parameter Configuration

```
aws docdb describe-db-cluster-parameters \
  --db-cluster-parameter-group-name <parameter-group-name>
```

List Cluster Snapshots

```
aws docdb describe-db-cluster-snapshots
```

Check Snapshot Sharing Attributes (Public Access)

```
aws docdb describe-db-cluster-snapshot-attributes \
  --db-cluster-snapshot-identifier <snapshot-id>
```

List Subnet Groups

```
aws docdb describe-db-subnet-groups
```

List Tags on a Cluster

```
aws docdb list-tags-for-resource \
  --resource-name arn:aws:rds:<region>:<account-id>:cluster:
  <cluster-id>
```

List Events for a Cluster

```
aws docdb describe-events \
  --source-type db-cluster \
  --source-identifier <cluster-id>
```

List Parameter Groups

```
aws docdb describe-db-cluster-parameter-groups
```

Check Engine Versions

```
aws docdb describe-db-engine-versions \
  --engine docdb
```

List Pending Maintenance Actions

```
aws docdb describe-pending-maintenance-actions
```

👤 Data Exfiltration Paths

#	Path	Method
1	Snapshot Share to External Account	<code>aws docdb modify-db-cluster-snapshot-attribute --db-cluster-snapshot-identifier <snap> attribute-name restore --values-to-add account-id</code> — grants restore access to a specific AWS account.
2	Snapshot Made Public	<code>aws docdb modify-db-cluster-snapshot-attribute --db-cluster-snapshot-identifier <snap> attribute-name restore --values-to-add public-access</code> — allows any AWS account can restore and read the data.
3	Cross-Region Snapshot Copy	<code>aws docdb copy-db-cluster-snapshot --db-cluster-snapshot-identifier <snap-arn> --target-db-cluster-snapshot-identifier <new-name> --key-id <attacker-key></code> — moves data to a region the attacker may not monitor.
4	Restore Snapshot to New Cluster	<code>aws docdb restore-db-cluster-from-snapshot --db-cluster-identifier <new-cluster> --snapshot-identifier <snap> --engine docdb</code> — spins up a new cluster from the snapshot of the database. Attacker connects with the original credentials.
5	Direct Query via MongoDB Driver	With network access and valid credentials, an attacker can use <code>mongosh</code> or any MongoDB driver to connect on port 27020 and run <code>db.collection.find()</code> to extract documents.



Detection — CloudTrail Events to Monitor

Event	Significance
CreatedDBClusterSnapshot	Manual snapshot creation — potential precursor to exfiltration
ModifyDBClusterSnapshotAttribute	Snapshot sharing changed — check if shared publicly or to unknown accounts
CopyDBClusterSnapshot	Snapshot copied — check target region and KMS key
RestoreDBClusterFromSnapshot	New cluster from snapshot — verify this is authorized
ModifyDBCluster	Check for deletion protection disabled, backup retention reduced
ModifyDBClusterParameterGroup	Check for <code>tls=disabled</code> or <code>audit_logs=disabled</code>
DeletedDBCluster	Cluster deletion — verify deletion protection was properly enforced
CreatedDBInstance (with docdb engine)	New instance added — could indicate unauthorized scaling or access

Key DocumentDB Cluster Parameters Reference

Parameter	Default	Allowed Values
<code>tls</code>	enabled	disabled, enabled, fips-140-tls1.2+, tls1.3+
<code>audit_logs</code>	disabled	enabled, disabled
<code>profiler</code>	disabled	enabled, disabled
<code>profiler_threshold_ms</code>	100	50 - 2147483647
<code>profiler_sampling_rate</code>	1.0	0.0 - 1.0
<code>change_stream_log_retention_duration</code>	10800	3600 - 604800
<code>ttl_monitor</code>	enabled	enabled, disabled

Static parameters require a manual reboot of every instance in the effect.

Security Hub Controls Summary

Control	Check	Severity
DocumentDB.1	Encryption at rest enabled	Medium
DocumentDB.2	Backup retention $\geq$ 7 days	Medium
DocumentDB.3	Manual snapshots not public	Critical
DocumentDB.4	Audit logs exported to CloudWatch	Medium
DocumentDB.5	Deletion protection enabled	Medium
DocumentDB.6	TLS encryption in transit enforced	Medium

Policy Examples

✗ Bad Policy — Overly Permissive

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "rds:*",
      "Resource": "*"
    }
  ]
}
```

Why this is dangerous: DocumentDB uses the `rds:` IAM action namespace. Granting `rds:*` gives full control over all RDS and DocumentDB resources — including creating snapshots, sharing them publicly, disabling deletion protection, and deleting clusters.

✓ Good Policy — Least Privilege Read-Only

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "rds:DescribeDBClusters",
        "rds:DescribeDBInstances",
        "rds:DescribeDBClusterParameters",
        "rds:DescribeDBClusterSnapshots",
        "rds:DescribeDBClusterSnapshotAttributes",
        "rds:DescribeDBSubnetGroups",
        "rds:ListTagsForResource"
      ],
      "Resource": "arn:aws:rds:*:123456789012:cluster:my-docdb-*"
    }
  ]
}
```

✗ Deny Dangerous Actions — Guardrail Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyDocDBSnapshotPublicSharing",
      "Effect": "Deny",
      "Action": "rds:ModifyDBClusterSnapshotAttribute",
      "Resource": "arn:aws:rds:*:*:cluster-snapshot:*"
    }
  ]
}
```



Defense Recommendations

Enforce TLS with Minimum Version

Use a custom parameter group with `tls=tls1.2+` or `tls=tls1.3+`. Never use `tls=enabled` (allows TLS 1.0). Reboot all instances after changing this static parameter.

```
aws docdb modify-db-cluster-parameter-group \  
  --db-cluster-parameter-group-name my-docdb-params \  
  --parameters "ParameterName=tls,ParameterValue=tls1.2+,ApplyMethod=pending-reboot"
```

Enable Full Audit Logging

Enable DDL and DML auditing and export to CloudWatch Logs:

```
aws docdb modify-db-cluster-parameter-group \  
  --db-cluster-parameter-group-name my-docdb-params \  
  --parameters "ParameterName=audit_logs,ParameterValue=enabled,ApplyMethod=immediate"
```

```
aws docdb modify-db-cluster \  
  --db-cluster-identifier my-cluster \  
  --cloudwatch-logs-export-configuration '{"EnableLogTypes":["audit","profiler"]}'
```

Enable Profiler for Slow Query Detection

```
aws docdb modify-db-cluster-parameter-group \  
  --db-cluster-parameter-group-name my-docdb-params \  
  --parameters "ParameterName=profiler,ParameterValue=enabled,ApplyMethod=immediate" \  
  "ParameterName=profiler_threshold_ms,ParameterValue=100,ApplyMethod=immediate"
```

Enable Deletion Protection

```
aws docdb modify-db-cluster \  
  --db-cluster-identifier my-cluster \  
  --deletion-protection
```

Block Public Snapshot Sharing via SCP

Apply this Service Control Policy at the AWS Organizations level:

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Sid": "DenyPublicDocDBSnapshots",  
      "Effect": "Deny",  
      "Action": "rds:ModifyDBClusterSnapshotAttribute",  
      "Resource": "*"  
    }  
  ]  
}
```

Use Customer Managed KMS Keys

Create clusters with a customer managed KMS key (not the default `aws/rds` key) to enable custom key policies and cross-account access control for encrypted snapshots.

Rotate Master Credentials via Secrets Manager

Configure Secrets Manager automatic rotation for the DocumentDB master password. Do not embed credentials in application code.

Restrict Security Group Access

Limit inbound port 27017 to specific application security groups only. Do not use CIDR-based rules within the VPC.

Set Backup Retention to at Least 7 Days

```
aws docdb modify-db-cluster \  
  --db-cluster-identifier my-cluster \  
  --backup-retention-period 7
```

Deploy AWS Config Rules

Enable the following AWS Config managed rules: - docdb-cluster-audit-logging-enabled - docdb-cluster-snapshot-public-prohibited - docdb-cluster-encryption-enabled (custom rule or Security Hub control DocumentDB.1)